

C++ Build Systems, STL Algorithms, and Shared Ownership

Grounded in ECM2433 cppIntro, L10, L13, and L16: C++ compile/link flow, CMake as a large-project build tool, STL algorithms/function objects, and shared/weak smart pointers.

What You Need To Know

1. C++ compile and link flow

C++ uses the same broad compile/link model as C, but use the C++ compiler driver. Source files such as `main.cpp` and `sensor.cpp` compile to object files. The object files are then linked into an executable.

```
g++ -std=c++17 -c main.cpp -o main.o
g++ -std=c++17 -c sensor.cpp -o sensor.o
g++ main.o sensor.o -o program
```

The output executable is usually named without `.o`. The `.o` extension means object file, not finished program.

2. CMake at a high level

Lecture L10 mentions `cmake` as the large-project build tool for C++. CMake does not replace the compiler. It generates build files for tools such as Make or Ninja, then those tools run the compile and link commands.

```
cmake -S . -B build
cmake --build build
```

- `-S .` says where the source project is.
- `-B build` says where generated build files and compiled output go.
- CMake is useful when a project has many files, libraries, include directories, and platform-specific build settings.

3. STL algorithms and iterators

STL algorithms work with iterator ranges. A common pattern is `container.begin()` and `container.end()`. `end()` is one past the last element.

```
#include <algorithm>
#include <vector>

std::vector<int> values = {4, 1, 7, 1};

std::sort(values.begin(), values.end());
bool has_four = std::binary_search(values.begin(), values.end(), 4);
int ones = std::count(values.begin(), values.end(), 1);
```

- `sort` reorders the range.
- `binary_search` assumes the range is already sorted according to the same ordering.
- `count` returns how many elements compare equal to a value.

4. Function objects and lambdas

Algorithms can be customised by passing a comparator or predicate. That callable can be a function, a function object, or a lambda.

```
std::sort(values.begin(), values.end(),
    [](int a, int b) {
        return a > b; // descending
    });

int big = std::count_if(values.begin(), values.end(),
    [](int value) {
        return value > 10;
    });
```

A sort comparator should return true when the first argument should come before the second. A count/filter predicate returns true for values that should be counted or kept.

5. `shared_ptr` and `weak_ptr`

`std::unique_ptr<T>` has one owner and cannot be copied. `std::shared_ptr<T>` is for shared ownership: copies increase a reference count, and the object is destroyed when the last shared owner goes away.

```
#include <memory>

auto p = std::make_shared<int>(42);
auto q = p; // both share ownership

std::cout << p.use_count(); // usually 2 here
```

`std::weak_ptr<T>` observes an object owned by shared pointers without increasing the strong ownership count. To use the object, call `lock()`. If the object is already gone, `lock()` returns an empty shared pointer.

```
std::weak_ptr<int> weak = p;

if (auto alive = weak.lock()) {
    std::cout << *alive << std::endl;
}
```

Practice Questions

Question 1 - finish the C++ build:

```
g++ -std=c++17 -c main.cpp -o main.o
g++ -std=c++17 -c sensor.cpp -o sensor.o
/* link command here */
```

- 1 Write the command that creates an executable called program.

Question 2 - explain the output file names:

```
g++ main.o sensor.o -o program
```

- 2 Why is the output called program rather than program.o?

Question 3 - CMake role:

```
cmake -S . -B build
cmake --build build
```

- 3 Explain what CMake is doing at a high level and whether it replaces g++.

Question 4 - binary search bug:

```
std::vector<int> values = {8, 2, 5, 1};
bool found = std::binary_search(values.begin(), values.end(), 5);
```

- 4 Explain the bug and fix the code.

Question 5 - complete descending sort:

```
std::vector<int> scores = {4, 10, 7};
std::sort(scores.begin(), scores.end(),
    /* comparator here */);
```

- 5 Complete the comparator so the largest score comes first.

Question 6 - choose the algorithm:

```
std::vector<int> xs = {1, 4, 4, 8};
/* count how many values are 4 */
```

- 6 Write the STL algorithm call that gives the count.

Question 7 - complete count_if:

```
std::vector<int> xs = {3, 12, 5, 20};
int n = std::count_if(xs.begin(), xs.end(),
    /* predicate here */);
```

- 7 Complete the predicate so n counts values greater than 10.

Question 8 - shared ownership trace:

```
auto a = std::make_shared<int>(5);
auto b = a;
auto c = b;
```

- 8 After these lines, explain how many shared owners there are and when the int can be destroyed.

Question 9 - weak_ptr access:

```
std::weak_ptr<int> observer = a;
/* safely print the int if it still exists */
```

- 9 Write the safe access pattern using `lock()`.

Question 10 - choose the pointer type:

```
// A tree node has one owning parent pointer to each child.
// A cache entry may be referenced by several systems.
// A back pointer observes an object owned elsewhere.
```

- 10 For each situation, choose `unique_ptr`, `shared_ptr`, or `weak_ptr` and explain the ownership reason.

Mark Scheme

Attempt all questions before checking.

- 1 `g++ main.o sensor.o -o program.`
- 2 `program.o` would mean another object file. The link step produces the final executable, so it is normally named `program`.
- 3 CMake configures/generates build files and drives a build tool. It does not replace the compiler; the generated build still runs compiler/linker commands such as `g++`.
- 4 `binary_search` requires a sorted range. Fix: `std::sort(values.begin(), values.end());` before the search.
- 5 Expected: `[](int a, int b) { return a > b; }.`
- 6 `int n = std::count(xs.begin(), xs.end(), 4);.`
- 7 Expected: `[](int value) { return value > 10; }.`
- 8 There are three shared owners. The managed `int` is destroyed after the last shared pointer owning it is destroyed or reset.
- 9 Expected: `if (auto p = observer.lock()) { std::cout << *p; }. lock()` checks whether the object still exists.
- 10 Tree child ownership: `unique_ptr`. Shared cache entry: `shared_ptr`. Non-owning back pointer: `weak_ptr` to avoid extending lifetime or creating ownership cycles.